

PROGRESS REPORT

Solving and Generating Sudoku as a Constraint Satisfaction Problem

Group Number: 3

Date: 07/21/2026

Team Members:

- Arjun Mohunta, amohunta, 80654131
- Haoyang Ding, haoyad6, 44024972
- Rajat Choudhary, rajatc1, 37222868

1. Project Summary

Our project treats Sudoku as a formal constraint satisfaction problem (CSP): 81 variables, one per grid cell, each with the domain {1 to 9}, subject to all different constraints across every row, every column, and every 3×3 box. The system has three goals, unchanged from our proposal: (1) solve any valid partial grid, including deliberately hard instances such as the 23 clue "AI Escargot" puzzle; (2) generate new puzzles at selectable difficulty levels with a verified unique solution; and (3) show the work, meaning we expose backtracking search and constraint propagation so a viewer can watch candidate domains shrink and dead ends unwind. To measure whether our techniques help, we also built a naive backtracking baseline and compared the two on search effort. The prototype described below already achieves a simplified version of all three goals.

2. Algorithm

2.1 CSP formulation and representation

We index cells 0 to 80 in row major order. For each cell we precompute its set of peers, the 20 other cells that share its row, column, or box, since these are exactly the cells whose assignments constrain it. Each cell's remaining candidate values are stored as a 9 bit integer mask, so domain operations reduce to fast bitwise arithmetic and a candidate count is a single population count lookup. This compact representation is what makes propagation cheap enough to run on every assignment.

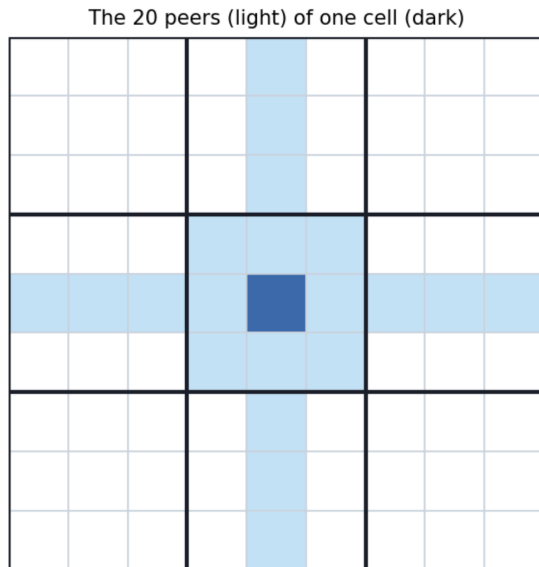


Figure 1. Every cell is constrained by the same 20 peers. Assigning a value prunes it from all 20 peer domains at once.

2.2 Smart solver: backtracking, forward checking, and MRV

The core solver is depth-first backtracking search augmented with two standard CSP techniques. First, forward checking constraint propagation: the moment we assign value v to cell i , we remove v from the candidate domain of every unassigned peer. Formally, for each peer p of i , the domain update is:

$$D_p \leftarrow D_p \setminus \{v\}$$

Where D_p is the candidate domain of cell p , meaning the set of values it can still legally take. If this leaves any D_p empty, the assignment cannot lead to a solution, so the solver backtracks and tries the next value for i . In other words, placing v in cell i deletes v from the option list of all 20 of its peers, and if that empties some peer's list, the guess is dead and we undo it.

Second, the Minimum Remaining Values (MRV) heuristic decides which cell to work on next. Instead of filling cells in reading order, the solver always expands the unassigned cell that currently has the fewest candidates, that is, the cell whose checklist has the fewest digits still ticked. This is the classic fail first principle: by branching on the most constrained cell, we either fill a cell that has only one option left or run into a contradiction quickly and with the smallest possible number of alternatives to explore. Combined with forward checking, an easy puzzle often solved by propagation alone, with zero backtracks. We chose not to add a value-ordering heuristic such as Least Constraining Value (LCV), which Russell and Norvig pair with MRV, because forward checking already prunes dead branches early enough that value order has little effect on our test instances; we may revisit this if the large-scale benchmark reveals cases where it helps.

Putting the two techniques together, the search runs recursively. If every cell is assigned, the puzzle is solved. Otherwise the solver picks the MRV cell and, for each value still in that cell's domain, tentatively assigns it, runs forward checking to prune the peers, and recurses. If a branch leads to a wipeout or fails later on, the solver undoes that assignment and its pruning and tries the next value. If no value works, it reports failure so the previous level can backtrack. Every pruning step is recorded so it can be undone exactly, which keeps the search sound and free of side effects.

2.3 Naive baseline

For comparison we implemented plain row major backtracking: find the first empty cell, try the digits 1 to 9 in order, test each against the row, column, and box, and recurse on the first that fits. It uses no propagation and no ordering heuristic, so on hard instances it can run for an intractable number of steps. We therefore bound it with a node cap (10,000,000 in our comparison harness), and hitting the cap is reported as "gave up." Because the only difference between this and the smart solver is forward checking and MRV, the comparison isolates exactly what those two techniques contribute.

2.4 Uniqueness test and generator

Generation reuses the search as a solution counter that stops as soon as it finds a second solution. To build a puzzle we first fill an empty grid with a random complete solution, then visit cells in random order and try removing each given, keeping a removal only if the puzzle still has exactly one solution. Removal continues until a clue count target for the requested difficulty is reached:

Difficulty	Target clues (givens)
easy	45
medium	34
hard	26

Because every kept removal is checked against the uniqueness counter, generated puzzles are correct by construction and always have one and only one solution.

3. Schedule, Milestones, Remaining Goals, and Challenges

3.1 Progress against the original milestones

Measured against the schedule in our proposal, the prototype has met the core milestones.

Milestone	Status	Evidence
1. Basic design: grid storage, input/output, test puzzles, simple solver (easy and medium)	Done	Bitmask grid, string and file parsing and printing, preset and escargot.txt puzzles, plain backtracking baseline
2. Core solver: rule out impossible values, fill most restricted cell first, handle hard puzzles	Done	Forward checking and MRV; Escargot 151 backtracks, 17 clue 4,048 backtracks, both in ms
3. Testing and metrics: time, backtracks, solve rate; smart vs. simple	Partly done	compare command and 13 case test suite on presets; large dataset solve rate and timing still pending
4. Visualization: step by step solve, show backtracks	Done	Terminal step by step animation
5. Final version: test on all puzzles, write up results, prepare demo	In progress	This report; full evaluation and demo to follow

The command line program is working: it takes an 81 character string, a preset name, or a text file, prints the solved grid plus solver statistics, and can animate the search step by step in the terminal, showing candidate marks shrink, highlighting the MRV cell before each guess, and marking assignments and dead ends as the solver backtracks. We also have a 13 case

automated test suite covering clue count targets, uniqueness, known solutions to the preset puzzles, and the naive solver's cap behavior.

3.2 Remaining goals and current limitations

Systematic evaluation. Move beyond the three preset puzzles to a larger, categorized benchmark set and report aggregate statistics with variance rather than single runs.

Difficulty calibration. Right now difficulty is defined purely by clue count, which correlates only loosely with how hard a puzzle actually is to solve, so we plan to also report solver effort (backtracks) as a difficulty signal.

Stronger propagation (stretch goal). Add constraint techniques beyond forward checking, for example arc consistency or naked and hidden single inference, and measure whether they reduce backtracks further on the hardest puzzles.

Compared with the final system we envision, the current one is still limited: evaluation rests on three puzzles, difficulty is only a proxy (clue count), and propagation stops at forward checking. None of these block the proof of concept; they are the improvements planned for the final report.

3.3 Anticipated challenges

The challenge we consider most significant is defining and validating difficulty rigorously. Hitting a clue count target is easy, but producing puzzles whose difficulty is meaningful and reproducible is harder, because two puzzles with the same clue count can differ by orders of magnitude in search effort. We plan to address this by generating many puzzles per level and reporting the distribution of backtracks, treating solver effort as the operational measure of difficulty. A second, milder challenge is worst case solve time on adversarial 17 clue puzzles; our solver handles the known hard cases in milliseconds, but a broad benchmark may surface slower instances, which stronger propagation should help with. We view both challenges as moderate and addressable within the remaining timeline.

4. Evaluation Plan

Our primary quantitative metric is search effort, measured as the number of backtracks needed to solve a puzzle. This directly captures the value of forward checking and MRV: fewer backtracks means the heuristics are pruning the search space effectively. We compare the smart solver against the naive baseline on the same puzzles, so any difference is due to the two techniques rather than to implementation details. Solve time in milliseconds is reported as a secondary metric.

Preliminary results, produced by `python3 demo.py compare` in this repository, already show a large and consistent gap. The `compare` command caps plain backtracking at 10 million nodes so the whole run finishes in about a minute; on the 17 clue puzzle plain search reaches that cap without solving, while the smart solver finishes in 4,048 backtracks.

Puzzle	Clues	Smart backtracks	Plain backtracks	Smart time
Easy	30	0	4,157	0.46 ms
AI Escargot	23	151	8,911	0.87 ms
Extreme (17 clue)	17	4,048	>10,000,000 (gave up)	14.22 ms

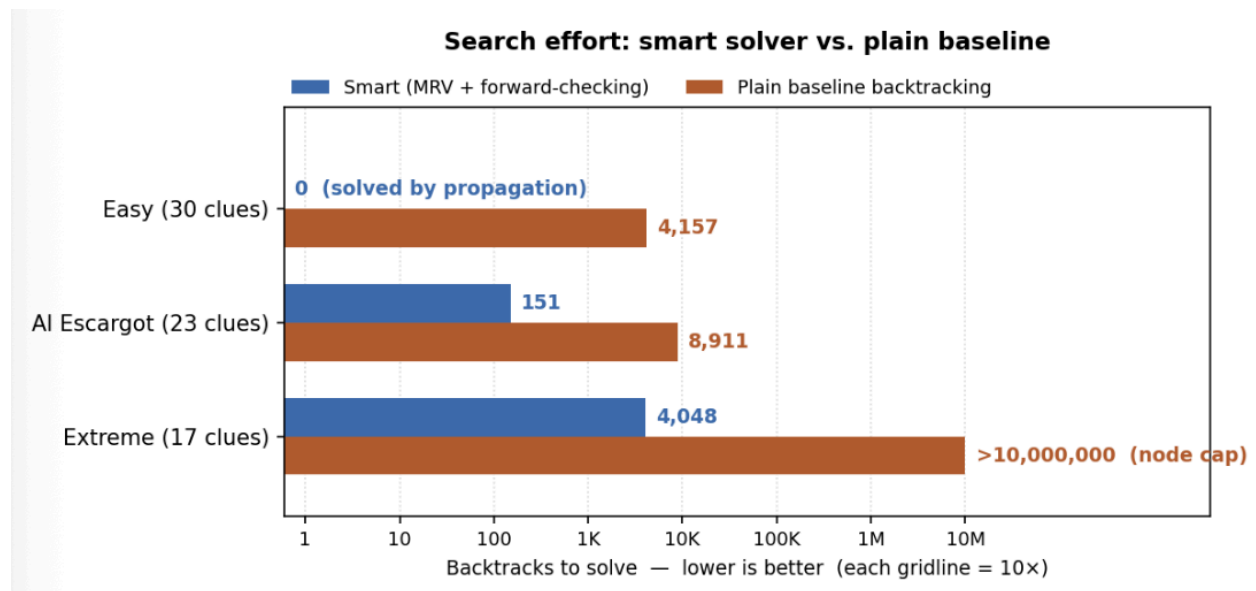


Figure 2. Backtracks to solve each puzzle (log scale; 0 backtracks shown as a label since $\log(0)$ is undefined). Plain backtracking hit the 10,000,000 node cap on the 17-clue puzzle without solving.

The takeaway is clear even at this stage: the smart solver solves the easy puzzle with zero backtracks, cuts AI Escargot from 8,911 backtracks to 151 (about a 59 times reduction), and solves the 17 clue puzzle in just 4,048 backtracks (about 14 ms), while plain backtracking passes 10,000,000 backtracks and gives up after roughly a minute without finding a solution, more than 2,400 times the work for no result. This meets and exceeds the 10 to 100 times reduction we predicted in the proposal.

Planned quantitative evaluation. We will scale this from three puzzles to the datasets named in our proposal: the Kaggle "1 million Sudoku games" set for broad solve rate and timing statistics, plus the well known top 95 hard puzzles and individual engineered cases such as AI Escargot for stress testing. We will report solve rate, the mean and spread of backtracks rather than single runs, and average solve time, and plot search effort against clue count. For the generator we will verify uniqueness on every generated puzzle using the solution counter and confirm reproducibility under fixed random seeds, both of which our test suite already covers and which we will report at scale.

Qualitative evaluation. The step by step terminal animation is our qualitative evidence that the solver reasons the way we claim: a viewer can watch the candidate marks shrink under propagation, see the MRV cell highlighted before each guess, and see dead ends appear as the solver backtracks. For the final report we will include annotated screenshots of representative solve traces, one showing a chain of forced cells on an easy puzzle and one showing a deep backtrack on a hard puzzle, to make the algorithm's behavior easy to follow.

```

Watching the CSP solver - givens=bold  placed=green  MRV  try  undo
+-----+-----+-----+
| 1 6 2 | 8 5 7 | 4 9 3 |
| 5 3 4 | 1 2 9 | 6 7 8 |
| 7 8 9 | 6 4 3 | 5 2 1 |
+-----+-----+-----+
| 4 7 5 | 3 1 2 | 9 8 6 |
| 9 1 3 | 5 8 6 | 7 4 2 |
| 6 2 8 | 7 9 4 | 1 3 5 |
+-----+-----+-----+
| 3 5 6 | 4 7 8 | 2 1 9 |
| 2 4 1 | 9 3 5 | 8 6 7 |
| 8 9 7 | 2 6 1 | 3 5 4 |
+-----+-----+-----+

Solved & valid.  backtracks=151  nodes=210  time=1.72 ms

```

Figure 3. A completed solve in the terminal (AI Escargot): bold = givens, green = filled by the solver; the run reports it verified valid in 151 backtracks

5. Resources Used

The implementation is deliberately dependency light. The resources that mattered most were:

Python 3 standard library only for the core solver, generator, and command line tool, with no third party AI or ML packages. This keeps the CSP logic transparent and the demo trivially runnable (random, time, dataclasses).

Russell and Norvig, Artificial Intelligence: A Modern Approach, specifically the constraint satisfaction chapter, for the standard formulations of backtracking search, forward checking, and the MRV heuristic that our solver implements.

Course materials for CS 175

AI Escargot and other well known hard puzzles, used as standard stress test instances for the solver.

The Python unittest framework for the regression test suite, and matplotlib for the evaluation figures in this report.

Peter Norvig, "Solving Every Sudoku Puzzle" (<https://norvig.com/sudoku.html>), a well-known reference implementation of constraint propagation and depth-first search for Sudoku that informed our design of the bitmask candidate representation and peer-based pruning.

The source code for our project is available at <https://github.com/arjunmohunta/suduko>